

SOLID PRINCIPLES

SOLID Principles

SOLID principles are **5 important rules** for writing good object-oriented code.

These principles help you write software that is easy to **understand**, easy to **modify**, and easy to **maintain**.

By following SOLID, your code becomes **cleaner** and more **professional**.

SOLID Principles

S → Single Responsibility Principle (SRP)

O → Open/Closed Principle (OCP)

L → Liskov Substitution Principle (LSP)

I → Interface Segregation Principle (ISP)

D → Dependency Inversion Principle (DIP)

S - Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change, meaning it should do only one job or have only one responsibility.

Simple terms: "Don't make one class do everything. Each class should focus on one task only."

Example: A *User* class should only handle user-related data, while database operations should be handled by a separate *UserRepository* class.

Bad Example

User Class

get_user_info()



is_adult()



save_to_database()



delete_from_database()



Good Example

User Class

get_user_info()



is_adult()



UserRepository Class

save_to_database()



delete_from_database()



O – Open Close Principle (OCP)

Definition: Software entities (classes, modules, functions) should be open for extension but closed for modification.

Simple terms:

- **Open for extension** = You can add new features easily
- **Closed for modification** = Don't change existing working code

Example: Adding new payment methods (Credit Card, UPI, PayPal) without modifying the existing payment processing code.

L – Liskov Substitution Principle (LSP)

Definition: It states that objects of a parent class should be replaceable with objects of a child class without breaking the program.

Simple terms:

- If Child class **is-a** Parent class, then Child should work **exactly like** Parent
- You should be able to use Child class wherever Parent class is expected
- Child class should **not break** the behavior that Parent class promises

No class should be forced to implement methods it doesn't use.

Split large interfaces into **smaller**, more **specific ones**.

I – Interface Segregation Principle

Definition: The Interface Segregation Principle ensures that classes are not forced to implement methods they don't need.

Simple terms:

- Break large, general-purpose interfaces into smaller, more specific ones.

Benefits:

- Maintainability – Code is easier to maintain and update
- Flexibility – Classes can pick and choose what they need
- Testability – Easier to test because classes have only what they need

Key Rule: Classes should only depend on the methods they actually use.

D – Dependency Inversion Principle (DIP)

Definition: High-level modules (business logic) should not depend directly on low-level modules (specific implementations). Both should depend on abstractions (interfaces).

Simple terms:

- **High-level** = Main business logic (what your app does)
- **Low-level** = Specific implementations (database, email, etc.)
- **Abstraction** = Interface/contract between them